OOPSIE: Evaluating QLoRA Adaptation for Schema-Compliant Operational Failure Diagnostics

Onyx Prasai
onyxprasai@gmail.com

Abstract

Deploying instruction-tuned language models as programmatic middleware in automated software diagnostics is frequently hindered by structural output instability under zero-tolerance parsing. We present OOPSIE (Operational Output Parser & Structured Inference Engine), an open evaluation framework examining whether Quantized Low-Rank Adaptation (QLoRA) improves structured JSON schema compliance for operational diagnostic reporting, and whether such adaptation alters qualitative diagnostic performance. We trained a QLoRA adapter (r=16, alpha=32) on top of the public model google/gemma-4-12B-it using a multi-turn developer error-recovery corpus and 4-bit NormalFloat quantization.

In a pre-specified frozen benchmark of 40 held-out failure scenarios (160 total greedy-decoding passes), QLoRA adaptation improved strict JSON schema compliance from 0.0% in unadapted base-model conditions to 95.0% (38/40) with schema guidance and 97.5% (39/40) without schema guidance. The comparison between unadapted zero-shot base-model inference and adapted inference with schema guidance was statistically significant under an exact two-sided McNemar test (38 discordant pairs; b=0, c=38; p approximately 7.28e-12; Edwards continuity-corrected chi-squared=36.03, p approximately 1.95e-9). This result concerns programmatic schema compliance, not diagnostic correctness.

An author-adopted qualitative evaluation, based on AI-generated provisional ratings across all 160 completions, did not demonstrate a diagnostic-reasoning or technical-correctness advantage for adapted conditions over unadapted base-model conditions. Within this benchmark, QLoRA adaptation functioned primarily as a structural output-format intervention rather than an amplifier of diagnostic reasoning. The benchmark, protocol, datasets, evaluation bundle, and result archive are publicly available; deployed systems require schema validation and human oversight.

Keywords: Large Language Models, Parameter-Efficient Fine-Tuning, QLoRA, JSON Schema Conformance, Automated Program Repair, Operational Diagnostics.

1. Introduction

Modern instruction-tuned foundation language models possess extensive parametric knowledge regarding software engineering and debugging, as demonstrated on code generation benchmarks such as HumanEval (Chen et al., 2021) and repository-level issue resolution benchmarks such as SWE-bench (Jimenez et al., 2024). However, deploying general-purpose chat models as programmatic middleware in automated toolchains presents severe operational challenges. Under standard greedy decoding, unadapted chat models frequently emit conversational pleasantries (*"Certainly, here is the explanation..."*), enclose JSON payloads in Markdown code fences (```json ... ```), or introduce syntactic defects such as illegal escape sequences. When downstream tooling enforces zero-tolerance RFC 8259 JSON validation (Bray, 2017) and rigid schema conformance (JSON Schema Draft 2020-12; Wright et al., 2022a, 2022b), these formatting deviations cause immediate automated ingestion failure.

In this work, we present **OOPSIE** (Operational Output Parser & Structured Inference Engine), an open evaluation framework designed to investigate the research question: *Does QLoRA adaptation improve structured JSON/schema*

compliance for operational-failure diagnostic reporting, and does it improve qualitative diagnostic reasoning? We evaluate this question across 40 unseen failure scenarios (160 forward passes) and show that QLoRA achieves 95.0%–97.5% strict schema compliance ($p \approx 7.28 \times 10^{-12}$), while systematic qualitative evaluation demonstrates that parameter adaptation does not alter or improve underlying diagnostic reasoning.

2. Related Work

Structured Generation: Prior approaches enforce output structure primarily through decoding-time grammar constraints, such as finite-state automata (FSM) guidance (Willard & Louf, 2023) or context-free grammar (CFG) token masking (Ugare et al., 2025). While decoding-time masking guarantees syntax correctness, it imposes custom runtime dependencies and vocabulary masking overhead. Leinonen et al. (2023) observed that unadapted LLMs explaining programming errors frequently introduce conversational artifacts.
Parameter-Efficient Fine-Tuning: Low-Rank Adaptation (LoRA; Hu et al., 2022) and Quantized Low-Rank Adaptation (QLoRA; Dettmers et al., 2023) inject trainable rank-decomposition matrices into frozen weights. While PEFT has been evaluated on general code generation (Rozière et al., 2023; Li et al., 2023), its efficacy at inducing rigid 10-key schemas under zero-tolerance parsing remains sparsely benchmarked.
Automated Program Repair: Traditional APR encompasses fault localization, patch synthesis, and test-suite validation (Zhang et al., 2024; Hou et al., 2024). OOPSIE acts not as an autonomous repair agent, but as an advisory diagnostic parser translating raw system logs (Kent & Souppaya, 2006) into validated, machine-readable structures.

3. Method & Experimental Design

Diagnostic Schema: OOPSIE targets a 10-key canonical JSON schema (Draft 2020-12): schema_version, error_category, confidence, risk_level, summary, root_cause, suggested_fix, safety_warning, repair_prompt, and verification_steps (with additionalProperties: false).
Base Model & QLoRA: We trained a QLoRA adapter on top of google/gemma-4-12B-it (~11.95B dense parameters; Gemma Team, 2026; Google LLC, 2026; recorded as google/gemma-4-12b-it in historical artifacts) using 4-bit NormalFloat quantization, r=16, α=32 across all linear projections (42.6M trainable parameters; 0.35%). Training was run for 2 epochs (44 steps) on an NVIDIA Tesla T4 GPU (16 GB).
Four Conditions: Condition A (Base Zero-Shot + Schema), Condition B (Adapted + Schema), Condition C (Base 1-Shot + Schema), and Condition D (Adapted No-Schema). All 160 completions used deterministic greedy decoding (temperature=0.0, max_new_tokens=1536).

4. Benchmark Results

Table 1 summarizes programmatic parsing and schema compliance results across all 160 completions.

Condition	Raw JSON	Lenient JSON	Strict Schema	Latency (s)	Primary Failure Mode
A: Base Zero-Shot + Schema	0/40 (0.0%)	40/40 (100.0%)	0/40 (0.0%)	118.6 ± 12.7	Markdown code fences / prose
B: Adapted + Schema	38/40 (95.0%)	38/40 (95.0%)	38/40 (95.0%)	114.4 ± 11.5	Truncated by 1536 max tokens (2)
C: Base 1-Shot + Schema	0/40 (0.0%)	40/40 (100.0%)	0/40 (0.0%)	119.4 ± 9.8	Markdown code fences / prose
D: Adapted No-Schema	40/40 (100.0%)	40/40 (100.0%)	39/40 (97.5%)	99.9 ± 11.9	Omitted risk_level key (1)

Inferential Statistical Test: Comparing Condition A to Condition B yields 38 discordant pairs (b=0, c=38, n=38). Under the exact two-sided McNemar test, $p = 2 \times (0.5)^{38} \approx 7.276 \times 10^{-12}$. The Edwards continuity-corrected $\chi^2 = 36.0263$ ($p \approx 1.947 \times 10^{-9}$), and uncorrected McNemar $\chi^2 = 38.00$ ($p \approx 7.074 \times 10^{-10}$). We explicitly note that this

significance claim pertains strictly to zero-tolerance schema compliance on the 40 paired benchmark scenarios (unadjusted for multiple comparisons), and does not imply superiority in technical diagnostic correctness.

Latency Measurement Note: Omitting schema text (Condition D) reduced mean single-run latency by 14.54 seconds (-12.7%). Latency was recorded sequentially on an isolated NVIDIA Tesla T4 GPU with batch size 1 without padding; this is an operational observation of context length reduction rather than a hardware guarantee.

Table 2: Qualitative diagnostic reasoning dimensions across all 160 completions.

Evaluation Dimension	Condition A (Base)	Condition B (Adapted)	Condition C (1-Shot)	Condition D (No-Schema)
Technical Correctness	37 Pass / 3 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail
Root Cause Accuracy	38 Pass / 2 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Fix Actionability	38 Pass / 2 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Safety Appropriateness	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Repair Prompt Utility	39 Pass / 1 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail

5. Discussion & Limitations

The empirical findings demonstrate that within this benchmark, QLoRA adaptation functioned primarily as a structural output-format intervention rather than an amplifier of diagnostic reasoning. The unadapted base model already possesses deep domain knowledge but defaults to conversational Markdown formatting. Training a QLoRA adapter aligns output generation directly to RFC 8259 bare JSON. Because parameter adaptation does not improve underlying reasoning, operational deployments must maintain rigorous validation gates: (1) automated JSON schema validation boundaries, and (2) mandatory human-in-the-loop review before executing suggested commands in production.

Limitations: (1) The benchmark is restricted to 40 curated scenarios; (2) 40 evaluations were conducted under a blinded protocol, while 120 were author-adopted AI-assisted ratings; (3) Scenarios were evaluated in static contexts rather than live execution environments; (4) Generalization is established specifically for google/gemma-4-12B-it and the OOPSIE 10-key schema.

6. Reproducibility & Conclusion

All benchmark artifacts, scripts, and logs are publicly available in the OOPSIE repository at https://github.com/Onyx-Prasai/OOPSIE and GitHub Release v1.2.0. Verified SHA-256 hashes: Evaluation Bundle (259bd473d9d18e0b13b79591ca1e44b0439667131b9e3966468d044ca05e42c5); Public Result Archive (cf24eec1a5c3347ae826fec4b5cbfb7133a1c4ce92aace3eab2b048bdd37f4eb).

Conclusion: QLoRA adaptation successfully induces rigid JSON schema conformance for operational software diagnostics (improving strict compliance from 0.0% to 95.0%–97.5%, $p \approx 7.28 \times 10^{-12}$). Crucially, the qualitative evaluation did not identify a diagnostic-reasoning or technical-correctness advantage for the adapted conditions over the unadapted base-model conditions. Within this benchmark, QLoRA adaptation functioned primarily as a structural output-format intervention rather than an amplifier of diagnostic reasoning.

AI Use Disclosure

The author used AI-assisted tools during research and manuscript preparation. Initial qualitative ratings for the 160 benchmark completions were generated by an automated evaluation pipeline using a frozen rubric. The author reviewed key outputs, verification findings, and flagged evaluations, and takes full responsibility for the final work. The remaining provisional qualitative ratings were retained without case-by-case author review and are reported as author-adopted AI-assisted evaluations. AI tools were also used to assist with code development, documentation, and drafting.

References

[1] Hu et al. (2022) LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.

[2] Dettmers et al. (2023) QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS 2023*.

[3] Bray (2017) The JavaScript Object Notation (JSON) Data Interchange Format. *RFC 8259*.

[4] Wright et al. (2022a) JSON Schema: A Media Type for Describing JSON Documents. *IETF Draft 2020-12*.

[5] Wright et al. (2022b) JSON Schema Validation: A Vocabulary for Structural Validation. *IETF Draft 2020-12*.

[6] Willard & Louf (2023) Efficient Guided Generation for Large Language Models. *arXiv:2307.09702*.

[7] Ugare et al. (2025) SynCode: LLM Generation with Grammar Augmentation. *TMLR*.

[8] Leinonen et al. (2023) Using Large Language Models to Enhance Programming Error Messages. *ACM SIGCSE*.

[9] Zhang et al. (2024) A Survey of Learning-Based Automated Program Repair. *ACM TOSEM*.

[10] Hou et al. (2024) Large Language Models for Software Engineering: A Systematic Literature Review. *ACM TOSEM*.

[11] Gemma Team (2026) Gemma 4 Technical Report. *arXiv:2607.02770*.

[12] Google LLC (2026) google/gemma-4-12B-it Model Card and Repository. *Hugging Face*.

[13] Chen et al. (2021) Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.

[14] Jimenez et al. (2024) SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*.

[15] Rozière et al. (2023) Code Llama: Open Foundation Models for Code. *arXiv:2308.12950*.

[16] Li et al. (2023) StarCoder: may the source be with you! *arXiv:2305.06161*.

[17] Kent & Souppaya (2006) Guide to Computer Security Log Management. *NIST SP 800-92*.